Searching & Sorting

[CSE2010] Data Structures

Department of Data Science

Searching

Process of finding an item with specified properties from a collection of items

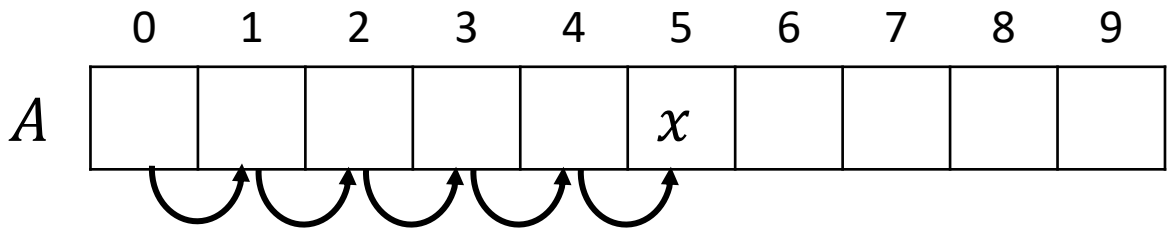
Search

- Given a sequence A of n elements, locate a given element x in A.
- When the array is unsorted
  - **Linear Search**
- When the array is sorted
  - **Binary Search**

	0	1	2	3	4	5	6	7	8	9
A						x				

Linear Search

- Sequential search
  1. Start from the leftmost element of A, and one by one compare x with each element of A.
  2. If x matches with an element, return the index of the element.
  3. If x does not match with any of elements, return a negative number, e.g., -1.



Linear Search

```
LinearSearch(A, x)
    for each element e in A
        if x matches with e
            return the position of e in A
    return -1
```

- Running time
  - Worst case: $O(n)$
  - Average case: $O(n)$
- Space complexity: $O(1)$
- Rarely used
  - Binary search and hash tables allow faster search.

Binary Search

- **Divide and conquer**
- Repeatedly divide the search interval in half
  1. Begin with an interval covering the whole array.
  2. If x is less than the item in the middle of the interval, narrow the interval to the lower half.
  3. Otherwise, narrow the interval to the upper half.
  4. Repeatedly check until x is found or the interval is empty.

	0	1	2	3	4	5	6	7	8	9
A						x				

Binary Search

- Find the position of $x = 15$: return 5

	0	1	2	3	4	5	6	7	8	9
A	3	4	7	9	10	15	17	21	23	29

$x > 10$ ⇓

	0	1	2	3	4	5	6	7	8	9
A	3	4	7	9	10	15	17	21	23	29

$x < 21$ ⇓

	0	1	2	3	4	5	6	7	8	9
A	3	4	7	9	10	15	17	21	23	29

$x = 15$

Binary Search (Recursive)

```
BinarySearchRec(A, x, low, high)
  if low > high
    return -1
  else
    mid ← ⌊(low + high)/2⌋
    if x == A[mid]
      return mid
    else if x < A[mid]
      return BinarySearchRec(A, x, low, mid − 1)
    else
      return BinarySearchRec(A, x, mid + 1, high)
```

- Suppose that array A is sorted
- Initially call **BinarySearchRec(**$A, x, 0, |A| - 1$**)**
- Running time: $T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$
  - Worst case: $O(logn)$
  - Average case: $O(logn)$
- Space complexity: $O(logn)$
  - The max depth of recursive function calls

Binary Search (Iterative)

```
BinarySearchIter(A, x)
  lower ← 0
  upper ← |A| − 1
  while lower ≤ upper
    mid ← ⌊(lower + upper)/2⌋
    if x == A[mid]
      return mid
    else if x < A[mid]
      upper ← mid − 1
    else
      lower ← mid + 1
  return -1
```

- Suppose that array A is sorted
- Running time
  - Worst case: $O(logn)$
  - Average case: $O(logn)$
- Space complexity: $O(1)$

Sorting

Arranging a sequence of elements in a certain order
(either ascending or descending).
The output is a permutation or reordering of the input.

Sorting Algorithms

- Bubble sort
- Shell sort
- Selection sort
- Insertion sort
- Merge sort
- Quicksort
- Heapsort
- Radix sort

⋮

A	8	2	6	7	1	5	10	4

↓

A	1	2	4	5	6	7	8	10

Selection Sort

- Maintain two subarrays in a given array
  - **Sorted subarray**
  - **Unsorted remaining subarray**
- For every iteration,
  1. Select the **minimum** element from the unsorted subarray
  2. Move it to the **sorted subarray**

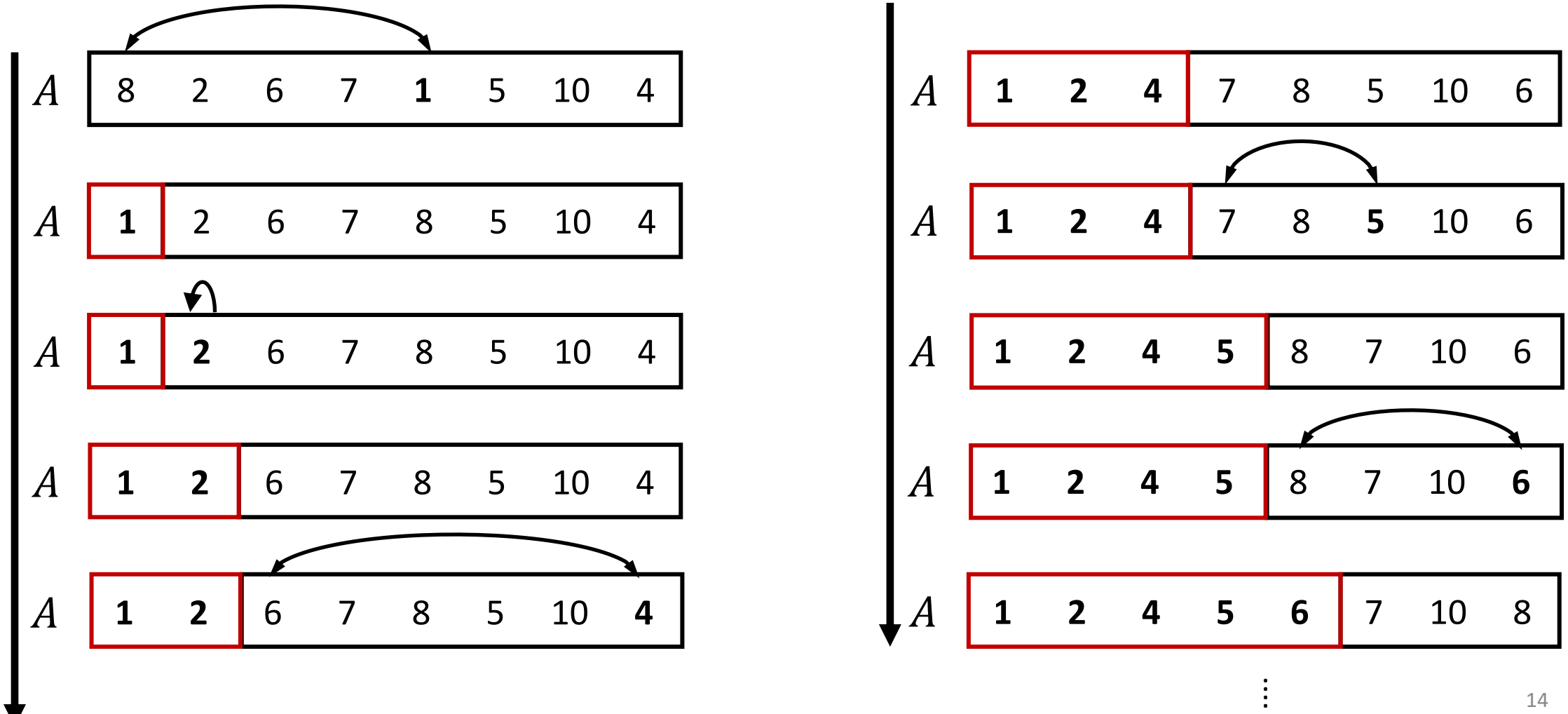
Implementation of Selection Sort

```
void SelectionSort(int A[], int n){
  for (int i = 0; i < n - 1; ++i) {
    int min = i;
    for (int j = i+1; j < n; ++j) {
      if (A[min] > A[j])
        min = j;
    }
    // Swap two elements
    int temp = A[min];
    A[min] = A[i];
    A[i] = temp;
  }
}
```

- Running time
  - $n + (n-1) + \cdots 2 + 1 = \frac{n(n+1)}{2} \in O(n^2)$
  - Best case: $O(n^2)$
  - **Worst case: $\boldsymbol{O(n^2)}$**
  - Average case: $O(n^2)$

- **In-place sort**
  - Space: $O(1)$

Selection Sort Example



Insertion Sort

- Maintain two subarrays in a given array
  - **Sorted subarray**
  - **Unsorted remaining subarray**
- For each iteration,
  1. Remove an element from the input
  2. Insert it into the correct position in the list being sorted
  3. Repeat until no input elements remain.

Implementation of Insertion Sort

```
void InsertionSort(int A[], int n){
  for (int i = 1; i < n; ++i) {
    int v = A[i];
    int j = i;
    while (A[j-1] > v && j >= 1) {
      A[j] = A[j-1];
      --j;
    }
    A[j] = v;
  }
}
```

- Running time
  - Best case: $O(n)$
  - **Worst case: $\boldsymbol{O(n^2)}$**
    - $1 + 2 + \cdots + n = \frac{n(n+1)}{2} \in O(n^2)$
  - Average case: $O(n^2)$
- **In-place sort**
  - Space: $O(1)$
- **Stable**

Insertion Sort Example

A	8	**2**	6	7	1	5	10	4
A	**2**	8	6	7	1	5	10	4
A	2	8	**6**	7	1	5	10	4
A	2	**6**	8	7	1	5	10	4
A	2	6	8	**7**	1	5	10	4

A	2	6	**7**	8	1	5	10	4
A	2	6	7	8	**1**	5	10	4
A	**1**	2	6	7	8	5	10	4
A	1	2	6	7	8	**5**	10	4
A	1	2	**5**	6	7	8	10	4

⋮

Divide and Conquer

1. **Divide** the problem into a number of subproblems that are smaller instances of the same problem.
2. **Conquer** the subproblems by solving them recursively.
   - If the subproblem sizes are small enough, solve the subproblems in a straightforward manner.
3. **Combine** the solutions to the subproblems into the solution for the original problem.

Merge Sort

- **Recursive** method
  - if n = 1, return (base case)
  - Otherwise, recursively merge-sort the first half and the second half (recursion), and then merge the two sorted halves.
- **Divide and conquer**
  1. **Divide**: divide the n-element array into two subarrays of n/2 elements each.
  2. **Conquer**: sort the two subarrays recursively using merge sort.
  3. **Combine**: merge the two sorted subarrays to produce the sorted answer.

	0	1	2	3	4	5	6	7
A	8	2	6	7	1	5	10	4

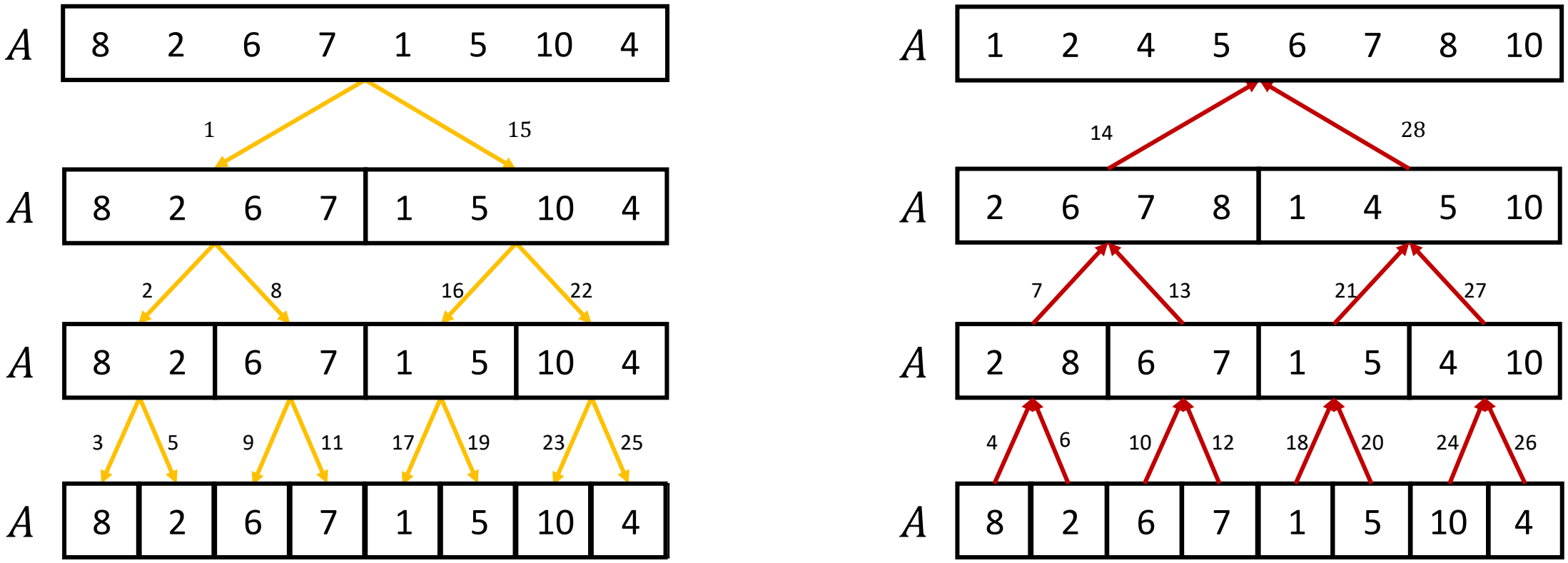
Merge Sort

```
MergeSort(A, p, r)
  if p < r
    q ← ⌊(p + r)/2⌋
    MergeSort(A, p, q)
    MergeSort(A, q + 1, r)
    Merge(A, p, q, r)
```

- Sort the elements in the subarray $A[p..r]$ (i.e., $A[p], \ldots, A[r]$)
- To sort entire sequence A, make the initial call: $\mathbf{MergeSort}(A, 0, |A| - 1)$

Merge Sort Example

- Divide-Conquer-Combine



Merge

```
Merge(A, p, q, r)
  n1 ← q − p + 1,   n2 ← r − q
  Let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
  for i = 1 to n1
    L[i] ← A[p + i − 1]
  for j = 1 to n2
    R[i] ← A[q + j]
  L[n1 + 1] ← ∞, R[n2 + 1] ← ∞   // sentinels
  i ← 1, j ← 1
  for k = p to r
    if L[i] ≤ R[j]
      A[k] ← L[i],  i ← i + 1
    else
      A[k] ← R[j], j ← j + 1
```

- Merge two sorted sequences in the "combine" step.
  - p, q, and r are indices into array A such that $p \leq q < r$.
  - Assumes that $A[p], \ldots, A[q]$ and $A[q+1], \ldots, A[r]$ are in sorted order.

Merge Example: $Merge(A, 0, |A|/2, |A|)$

A	2	6	7	8	1	4	5	10
T								

A	2	6	7	8	1	4	5	10
T	**1**							

A	2	6	7	8	1	4	5	10
T	1	**2**						

A	2	6	7	8	1	4	5	10
T	1	2	**4**					

A	2	6	7	8	1	4	5	10
T	1	2	4	**5**				

A	2	6	7	8	1	4	5	10
T	1	2	4	5	**6**			

Merge Example: $Merge(A, 0, |A|/2, |A|)$

A	2	6	7	8	1	4	5	10
T	1	2	4	5	6	**7**		

A	2	6	7	8	1	4	5	10
T	1	2	4	5	6	7	**8**	

A	2	6	7	8	1	4	5	10
T	1	2	4	5	6	7	8	**10**

Implementation of Merge Sort

```
void Merge(int A[], int T[], int lPos, int rPos, int
rEnd) {
  int lEnd = rPos - 1;
  int tPos = lPos;
  int size = rEnd - lPos + 1;
  while (lPos <= lEnd && rPos <= rEnd) {
    if (A[lPos] <= A[rPos])
      T[tPos++] = A[lPos++];
    else
      T[tPos++] = A[rPos++];
  }
  while (lPos <= lEnd)
    T[tPos++] = A[lPos++];
  while (rPos <= rEnd)
    T[tPos++] = A[rPos++];
  for (int i = 0; i < size; ++i, --rEnd)
    A[rEnd] = T[rEnd];
}
```

```
void MergeSort(int A[], int T[], int left, int
right) {
  if (left < right) {
    int mid = (left + right) / 2;
    MergeSort(A, T, left, mid);
    MergeSort(A, T, mid+1, right);
    Merge(A, T, left, mid+1, right);
  }
}
```

Recurrence of Divide-and-Conquer Algorithms

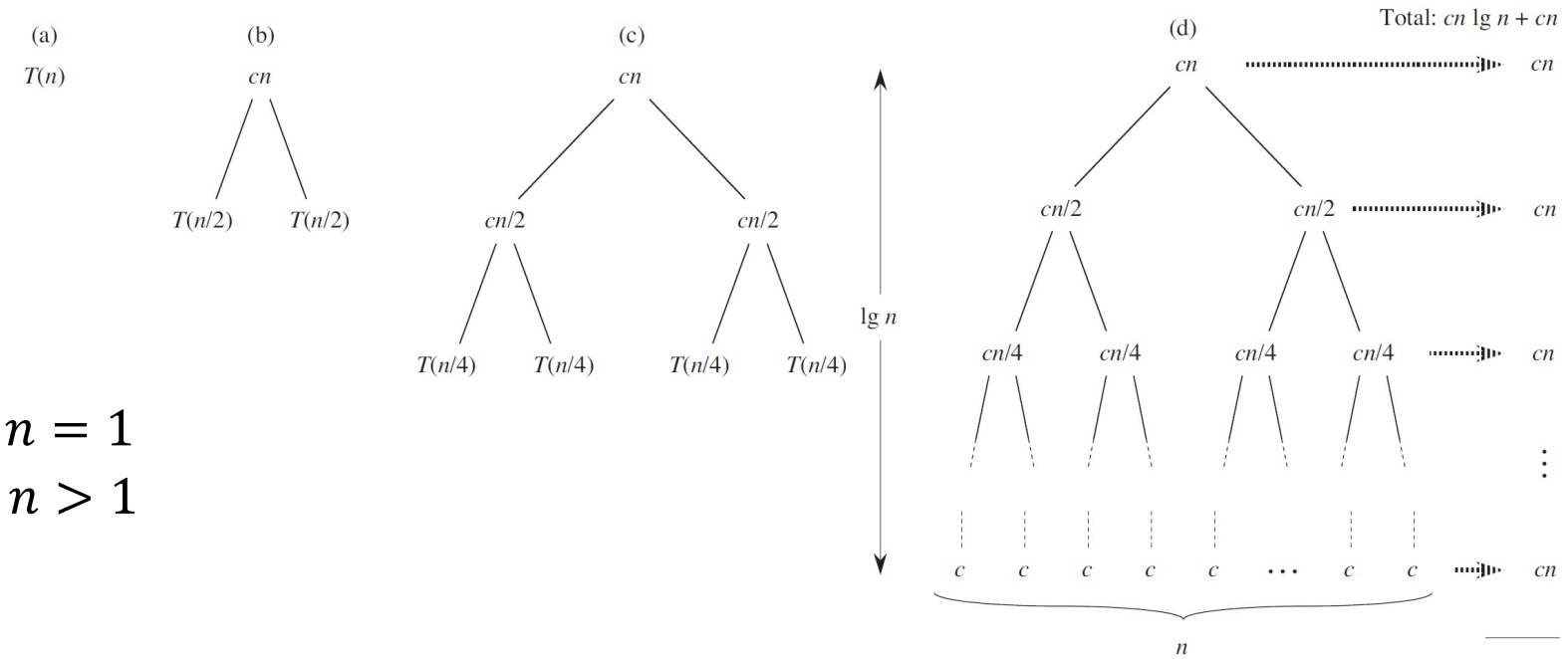
- Suppose that our division of the problem yields a subproblems, each of which is $1/b$ the size of the original.

$$T(n) = \begin{cases} \Theta(1) & if\ n \leq c \\ aT(n/b) + D(n) + C(n) & otherwise \end{cases}$$

  - $T(n)$: running time on a problem of size n
  - $D(n)$: time to divide the problem into subproblems
  - $C(n)$: time to combine the solutions to the subproblems in the solution to problem

Cormen, Leiserson, Rivest, and Stein. “Introduction to Algorithms” 3rd edition, MIT Press, 2009

Analysis on Merge Sort



Cormen, Leiserson, Rivest, and Stein. "Introduction to Algorithms" 3rd edition, MIT Press, 2009

- Recurrence of merge sort
- $T(n) = \begin{cases} c & if\ n = 1 \\ 2T(n/2) + cn & if\ n > 1 \end{cases}$
- $T(n) = 2T(n/2) + cn = 4T(n/4) + cn + cn = \cdots$
  $= \Sigma_{i=0}^{logn} cn = O(nlogn)$

Analysis on Merge Sort

- **Running time**
  - Best case: $O(nlogn)$
  - **Worst case: $\boldsymbol{O(nlogn)}$**
  - Average case: $O(nlogn)$
- **Not in-place sort**
  - Each recursive cell makes use of a temporary array, locally for each recursive call of MergeSort.
  - Space: $O(n)$ for auxiliary data structure
- **Stable sort**

Quicksort

- **Divide and conquer**
  - **Divide**: partition array $A[p..r]$ into two subarrays $A[p..q-1]$ and $A[q+1..r]$ such that $A[p..q-1] \leq A[q]$, and $A[q+1..r] > A[q]$.
  - **Conquer**: recursively sort the two subarrays $A[p..q-1]$ and $A[q+1..r]$.

- To sort array A, the initial call is $\mathbf{Quicksort}(A, 0, |A|-1)$

$\mathbf{Quicksort}(A, p, r)$

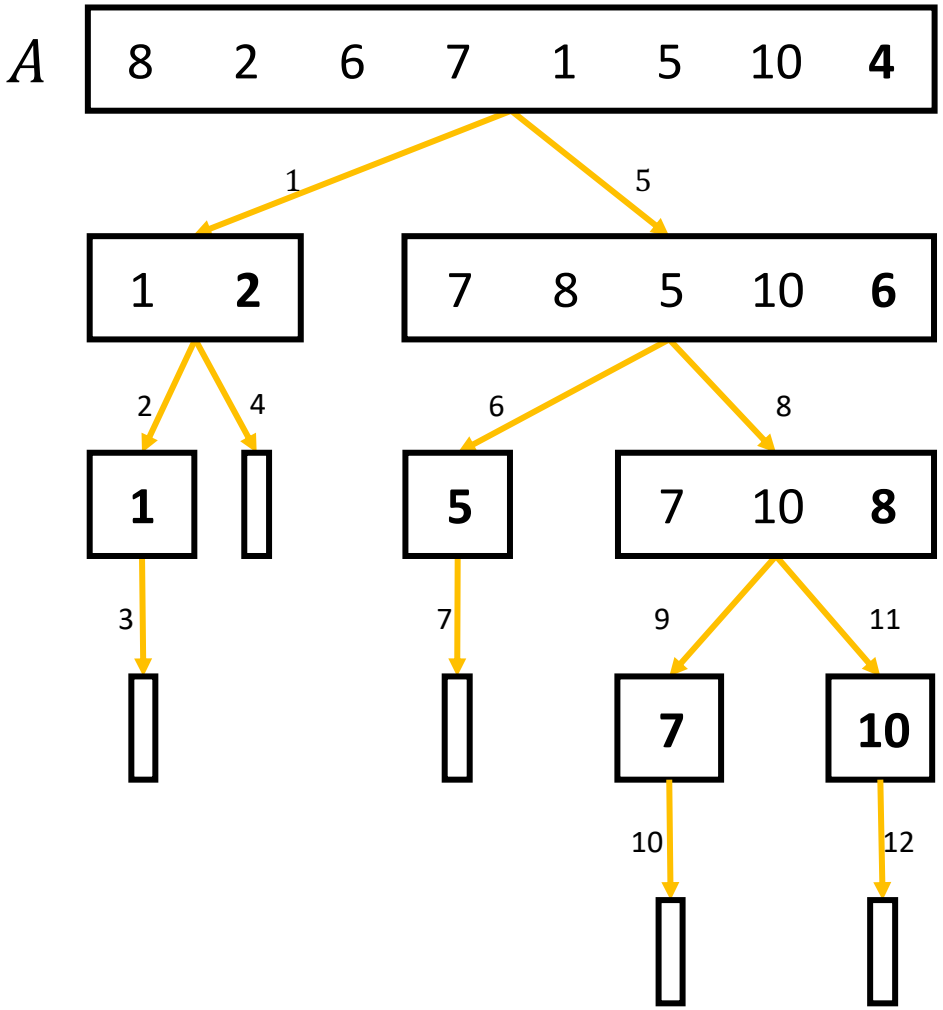
  $\mathbf{if}\ p < r$

    $q \leftarrow \mathbf{Partition}(A, p, r)$

    $\mathbf{Quicksort}(A, p, q-1)$

    $\mathbf{Quicksort}(A, q+1, r)$

Divide and Conquer in Quicksort



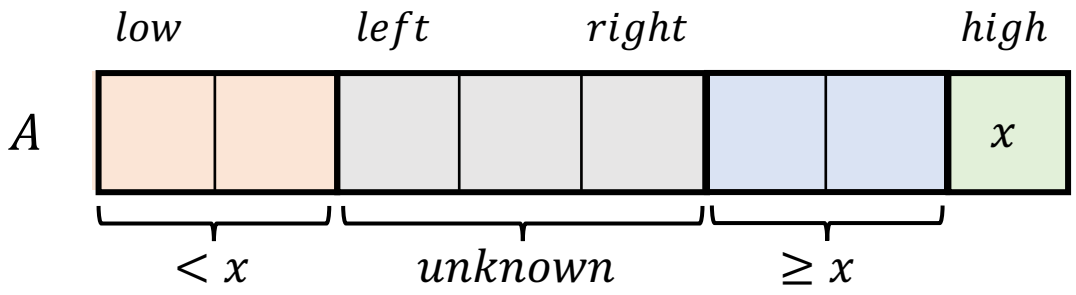
Partition

```
Partition(A, low, high)
  x ← A[high]  // Choose the rightmost element as pivot
  left ← low
  right ← high
  while left < right        // (*)
    while A[left] < x       // Move right while item <= pivot
      left ← left + 1
    while A[right] ≥ x      // Move left while item > pivot
      right ← right − 1
    if left < right         // ($)
      exchange A[left] with A[right]
  exchange A[left] with A[high]  // Move pivot to middle (#)
  return left
```

- For any $low \leq \boldsymbol{k} \leq high$ at while loop $(*)$

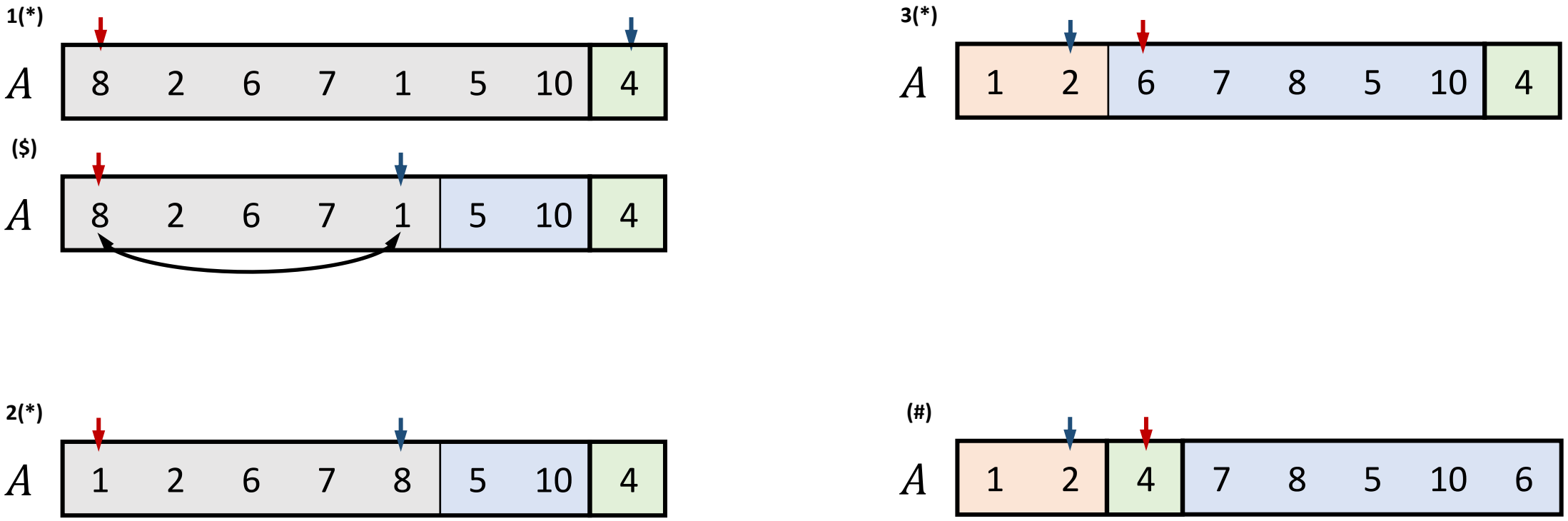
1. if $low \leq k < left$, then $A[k] < x$
2. if $right < k < high$, then $A[k] \geq x$
3. if $k = high$, then $A[k] = x$

- Running time: $\Theta(high - low + 1)$



Partition Example

Pivot



Quicksort Example

A	8	2	6	7	1	5	10	4
A	1	2	4	7	8	5	10	6
A	1	2	$\underline{4}$	7	8	5	10	6
A	1	2	$\underline{4}$	7	8	5	10	6
A	1	$\underline{2}$	$\underline{4}$	7	8	5	10	6

A	$\underline{1}$	$\underline{2}$	$\underline{4}$	7	8	5	10	6
A	$\underline{1}$	$\underline{2}$	$\underline{4}$	5	6	7	10	8
A	$\underline{1}$	$\underline{2}$	$\underline{4}$	5	$\underline{6}$	7	10	8
A	$\underline{1}$	$\underline{2}$	$\underline{4}$	$\underline{5}$	$\underline{6}$	7	10	8
A	$\underline{1}$	$\underline{2}$	$\underline{4}$	$\underline{5}$	$\underline{6}$	7	8	10
A	$\underline{1}$	$\underline{2}$	$\underline{4}$	$\underline{5}$	$\underline{6}$	7	$\underline{8}$	10
A	$\underline{1}$	$\underline{2}$	$\underline{4}$	$\underline{5}$	$\underline{6}$	$\underline{7}$	$\underline{8}$	10

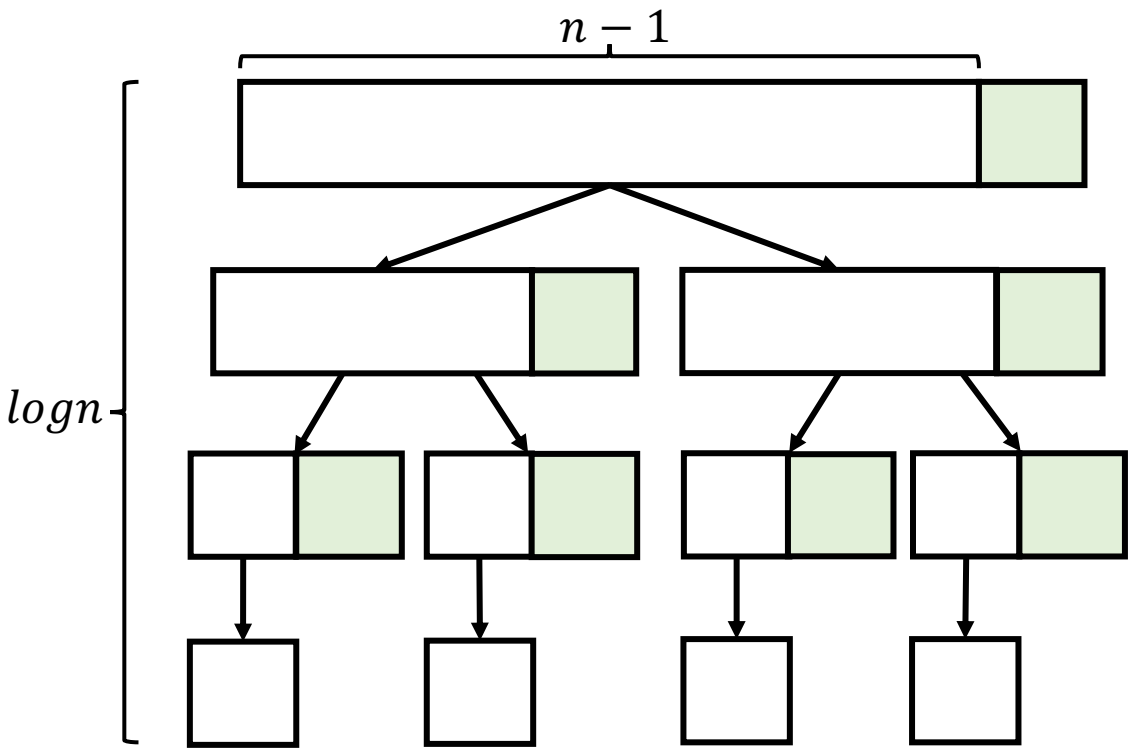
Quicksort Implementation

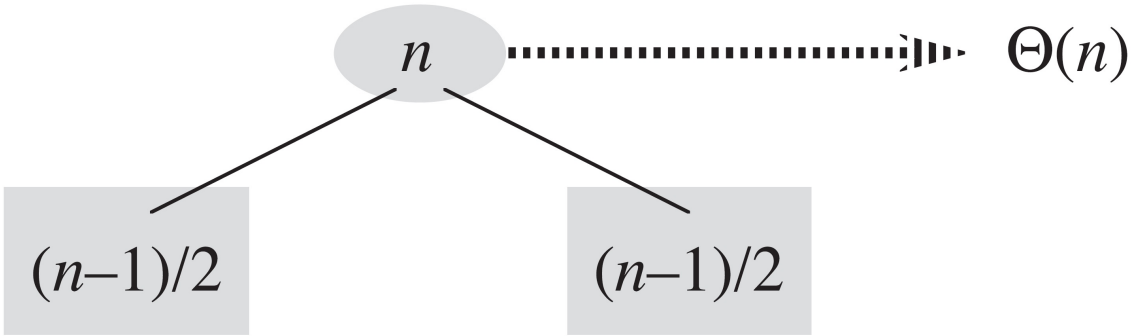
```
void Partition(int A[], int low, int high) {
  int pivot = A[high]; // Pick a pivot from A
  int left = low, right = high;
  while (left < right){
    while (A[left] < pivot)
      ++left;
    while (A[right] >= pivot)
      --right;
    if (left < right)
      swap(A, left, right);
  }
  A[high] = A[left];
  A[left] = pivot;
  return left;
}
```

```
void Quicksort(int A[], int low, int high) {
  if (low < high) {
    p = Partition(A, low, high);
    Quicksort(A, low, p-1);
    Quicksort(A, p+1, high);
  }
}
```

Recursion Tree of Quicksort in the Best Case

- Best case

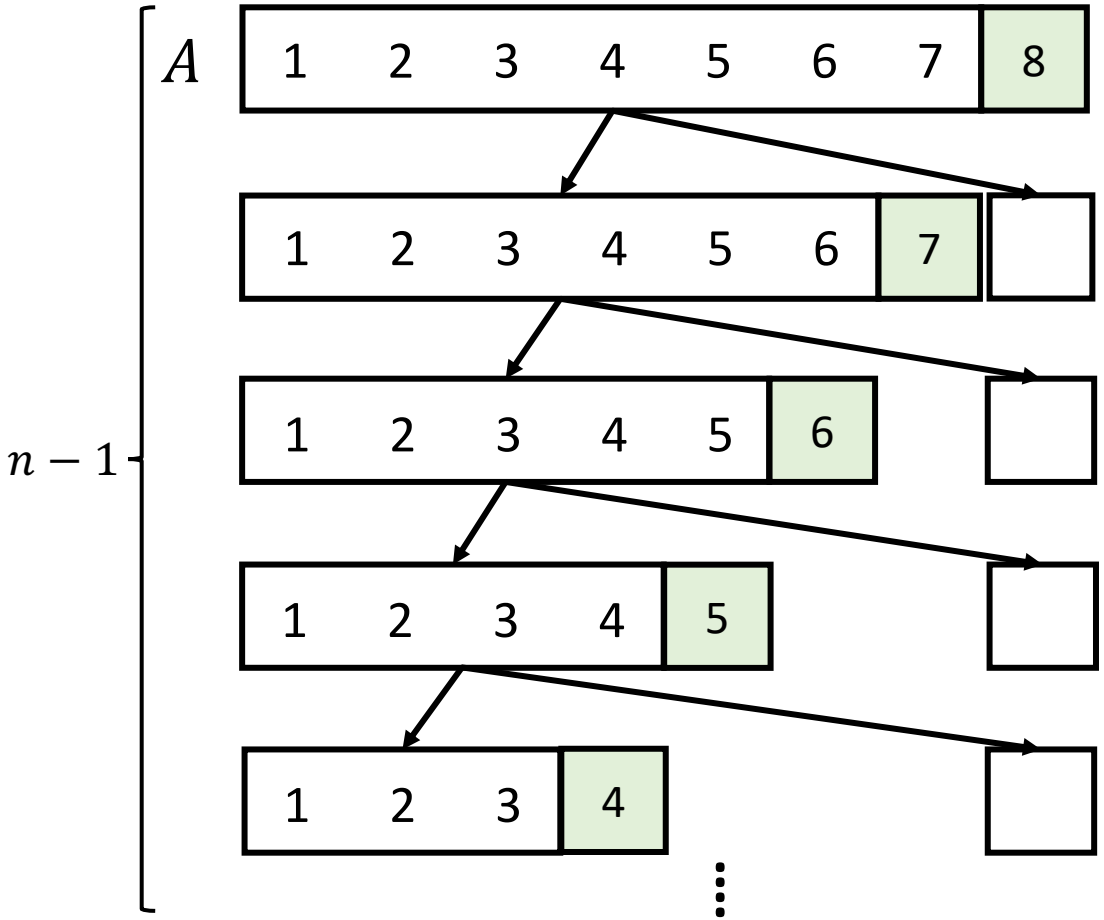


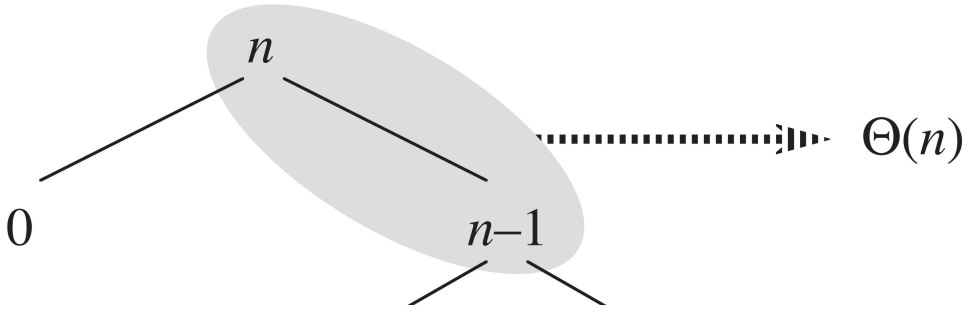


Cormen, Leiserson, Rivest, and Stein. “Introduction to Algorithms” 3rd edition, MIT Press, 2009

Recursion Tree of Quicksort in the Worst Case

- What if the array is already sorted?





Cormen, Leiserson, Rivest, and Stein. “Introduction to Algorithms” 3rd edition, MIT Press, 2009

Analysis on Quicksort

- $T(n) = \begin{cases} 0 & if\ n = 0\ or\ n = 1\ \ (base\ case) \\ T(i) + T(n-i-1) + cn & if\ n > 1\ \ (recursive\ case) \end{cases}$

Runtime of **Partition** for n elements

- Best case: divide into approximately n/2 and n/2 elements
- $T(n) = 2T(n/2) + n = 4T(n/4) + 2n = \cdots = \Sigma_{i=2}^{logn} n = O(nlogn)$

- Worst case: divide into n-1 elements and the pivot
- $T(n) = T(n-1) + n = T(n-2) + n - 1 + n = \cdots = T(1) + \Sigma_{i=2}^{n} i = O(n^2)$

Analysis on Quicksort: Average Case

- $T(n) = \begin{cases} 0 & if\ n = 0\ or\ n = 1 \quad (base\ case) \\ T(i) + T(n-i-1) + n & if\ n > 1\ \ (recursive\ case) \end{cases}$
- Assume that the size of partition is equally likely (probability is $1/n$)
- $T(n) = \frac{1}{n}\Sigma_{i=0}^{n-1}(T(i) + T(n-i-1)) + n = \frac{2}{n}\Sigma_{j=0}^{n-1}T(j) + n$
  - Average of $T(i)$ or $T(n-i-1)$ is $\frac{1}{n}\Sigma_{j=0}^{n-1}T(j)$
- $nT(n) = 2\Sigma_{j=0}^{n-1}T(j) + n^2$
- $(n-1)T(n-1) = 2\Sigma_{j=0}^{n-2}T(j) + (n-1)^2$
- $nT(n) - (n-1)T(n-1) = 2T(n-1) + 2n - 1$
- $nT(n) = (n+1)T(n-1) + 2n - 1 < (n+1)T(n-1) + 2n$
- $\frac{T(n)}{n+1} < \frac{T(n-1)}{n} + \frac{2}{n+1}$
- Harmonic series: for positive integers n, the nth harmonic number is $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \Sigma_{k=1}^{n}\frac{1}{k} = \ln n + O(1)$
- $\frac{T(n)}{n+1} < \frac{T(1)}{2} + 2\Sigma_{i=3}^{n+1}\frac{1}{i} = O(logn)$
- $T(n) = O(nlogn)$

Analysis on Quicksort

- Running time
  - Best case: $O(nlogn)$
  - **Worst case: $\boldsymbol{O(n^2)}$**
  - Average case: $O(nlogn)$
- Benefits
  - **In-place sort**
- Limitations
  - Use the last element as the pivot: worst if the input is sorted in the ascending or descending orders
- Alternatives
  - Median-of-three: choose the median of the leftmost, rightmost, and center elements
  - Randomized quicksort: choose the pivot randomly

Randomized Algorithms

- **Randomized algorithm**: behavior is determined not only by its input but also by values produced by a random number generator
- **Expected running time**: the running time of a randomized algorithm

- Difference from the average case
  - average-case running time: the probability distribution is over the inputs to the algorithm
  - expected running time: the algorithm itself makes random choices.

Randomized Quicksort

- The variation of quicksort
- Pick as a pivot a random element of the input sequence.
- **Expected running time: $O(nlogn)$**

RandomizedPartition(A, p, r) $i \leftarrow \text{Random}(p, r)$ exchange $A[r]$ with $A[i]$ **return** $\text{Partition}(A, p, r)$	**RandomizedQuicksort**(A, p, r) **if** $p < r$ $q \leftarrow$ **RandomizedPartition**(A, p, r) **RandomizedQuicksort**$(A, p, q-1)$ **RandomizedQuicksort**$(A, q+1, r)$)

Counting Sort

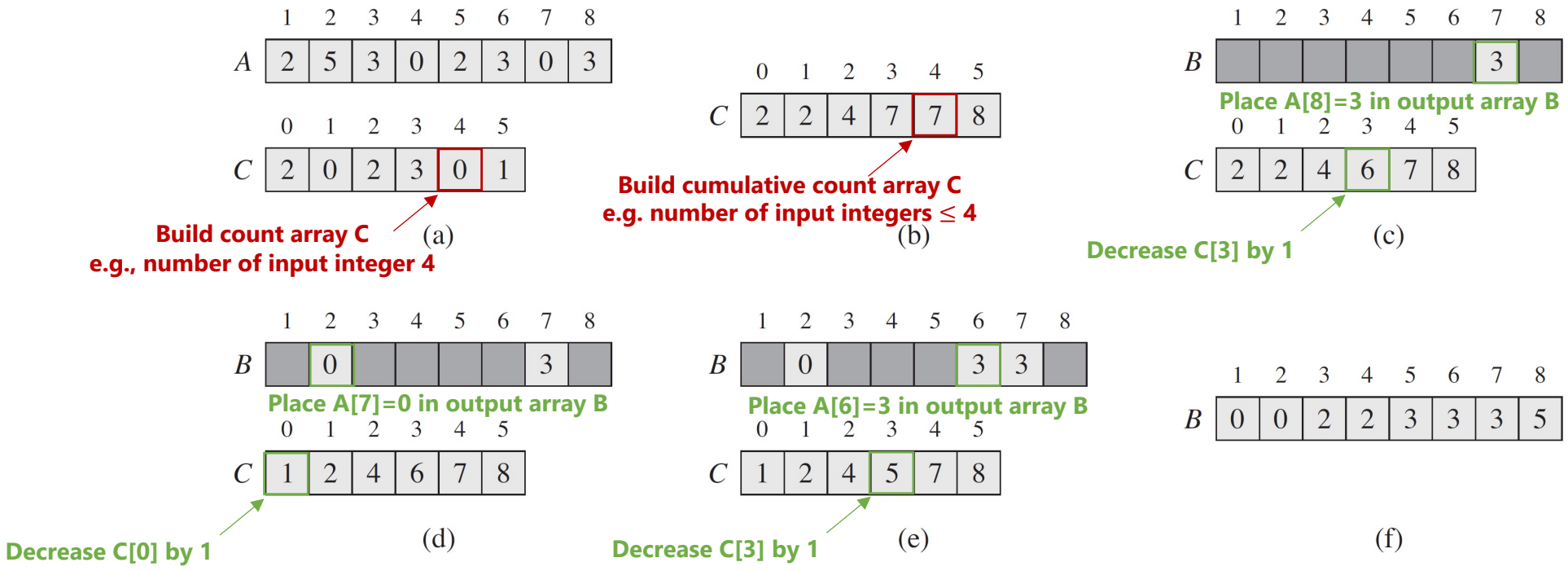
- Sort $\boldsymbol{n}$ **non-negative integers**
  - **Assume that each of the $\boldsymbol{n}$ input elements is an integer in [0, k], for some integer k**.
  - When k = O(n), the sort runs in $\boldsymbol{\Theta(n)}$ **time.**
- **Non-comparison sort**: no comparisons between input elements occur anywhere in the code.

```
CountingSort(A, k)
Θ(n)  Let B[1..n] be a new array // output array
Θ(k)  Let C[0..k] be a new array and initialize all elements with 0
Θ(n)  for i = 1 to |A|
          C[A[i]] ← C[A[i]] + 1  // C[j] contains the number of elements equal to j
Θ(k)  for j = 1 to k
          C[j] ← C[j] + C[j − 1]  // C[j] contains the number of elements <= j
Θ(n)  for i = |A| downto 1
          B[C[A[i]]] ← A[i]
          C[A[i]] ← C[A[i]] − 1
```

Running time: $\boldsymbol{\Theta(k+n) = \Theta(n)}$

Counting Sort on Array A[1,…,8]

- Each element of A is a nonnegative integer no larger than k = 5.
- **Stable sort**: numbers with the same value appear in the output array in the same order as they do in the input array.



Radix Sort

RadixSort(A, d)

for $j = d$ downto 1

Do a stable sort on A by jth digit.

- Sort n d-digit integers in array A
- **Non-comparison sort**
- **Stable sort**
  - Because stable sort (e.g., counting sort) is often used as a subroutine in radix sort.
- **Running time:** $\boldsymbol{O(nd) = O(n)}$
  - $\boldsymbol{d}$**: constant**

Radix Sort

0123	1560	0004	0004	0004
2154	2150	0222	1061	0123
0222	1061	0123	0123	0222
0004	0222	2150	2150	0283
0283	0123	2154	2154	1061
1560	0283	1560	0222	1560
1061	2154	1061	0283	2150
2150	0004	0283	1560	2154

Classification of Sorting Algorithms

- By Number of Comparisons
  - Comparison sort: the sorted order is based only on comparisons between the input elements
    - **Needs at least $\boldsymbol{O(nlogn)}$ comparisons**
    - Best case behavior is $O(nlogn)$
    - Worst case behavior is $O(n^2)$
  - Non-comparison: linear time, but impose restrictions on the input to improve the running time.
- By Memory Usage: in-place or not
  - **In-place**: need O(1) memory to create auxiliary space to sort elements.
- By Stability: stable or unstable
  - Sorting algorithm is **stable** if for all indices i and j such that key $A[i]$ equals key $A[j]$, the associated record $R[i]$ precedes record $R[j]$ in the sorted list if $R[i]$ precedes $R[j]$ in the input

Comparison of Different Sorting Algorithms

- Sorting algorithm animations
  - https://www.toptal.com/developers/sorting-algorithms

Type	Algorithm	Worst Case	Average Case	Auxiliary Space	Is Stable?
Comparison Sort	Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	No
	Insertion Sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
	Merge Sort	$O(nlogn)$	$O(nlogn)$	$O(n)$	Yes
	Quicksort	$O(n^2)$	$O(nlogn)$	$O(logn)$	No
	Heapsort	$O(nlogn)$	$O(nlogn)$	$O(1)$	No
	Radix Sort	$O(n)$	$O(n)$	$O(n)$	Yes

Selection

Select the i-th smallest element from an unsorted collection of n elements (the selection problem)

How to Solve the Selection Problem?

- How?

1. Sort the input sequence
2. Get the the i-th element from the sorted array

- **Running time: $O(nlogn)$**

$i = 3$

	0	1	2	3	4	5	6	7
A	8	2	6	7	1	5	10	4

	0	1	2	3	4	5	6	7
A	1	2	**4**	5	6	7	8	10

$i = 3$

$\mathbf{4}$

Prune-and-Search (Decrease-and-Conquer)

1. Prune away a fraction of the n objects
2. Recursively solve the smaller problem

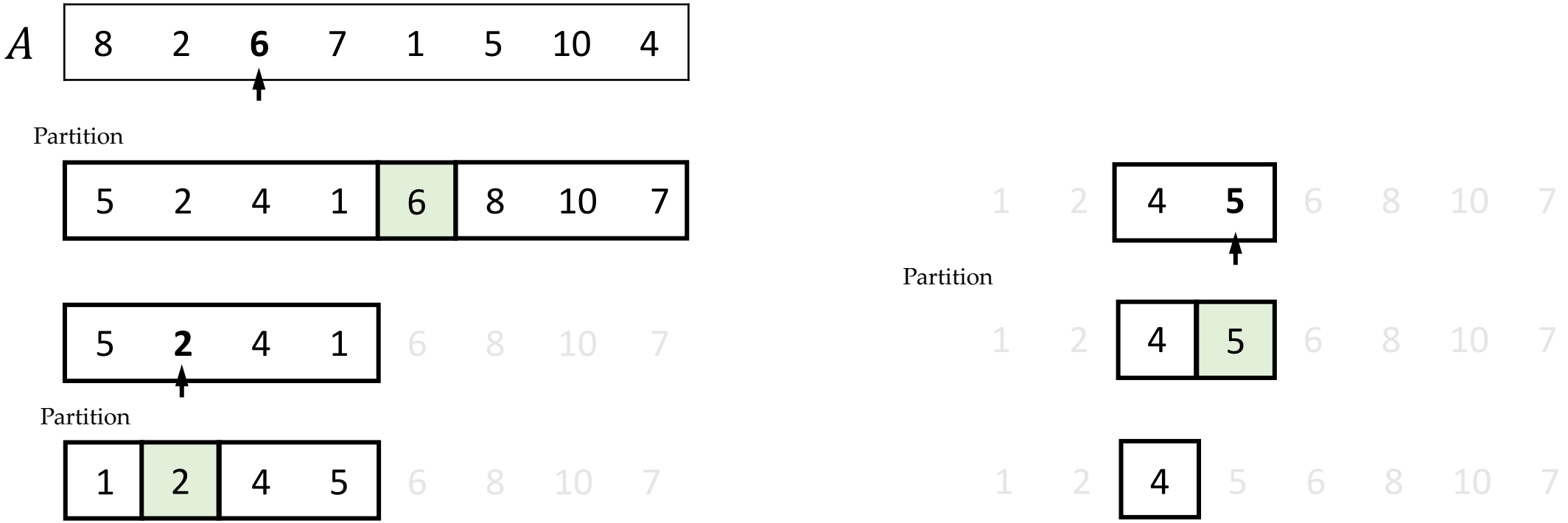
- Examples
  - Binary search
  - **Randomized (quick-)select**

Randomized Select

```
RandomizedSelect(A, p, r, i)
  if p == r
    return A[p]
  q ← RandomizedPartition(A, p, r) // O(n)
  k ← q − p + 1
  if i == k // the pivot value is the answer
    return A[q]
  else if i < k
    return RandomizedSelect(A, p, q − 1, i)
  else
    return RandomizedSelect(A, q + 1, r, i − k)
```

- Works on only one side of the partition.
  - Unlike quicksort that recursively processes both sides of the partition
- **Worst case running time: $O(n^2)$**
  - Partition around the largest remaining element
- **Expected running time: $O(n)$**

Find the 3rd smallest element in A



Exercise 1

- Given an array $A[0..n-1]$ of n numbers containing the repetition of some number.
- Give an algorithm for checking whether there are repeated elements or not.
  - Assume that we are not allowed to use additional space
    (we can use a few temporary variables in $O(1)$ space).
- Example 1
  - Input: [5, 7, 1]
  - Output: False
- Example 2
  - Input: [5, 7, 1, 5]
  - Output: True

Exercise 1

```
int checkDuplicates(int A[], int n) {
  for (int i = 0; i < n; ++i) {
    for (int j = i+1; j < n; ++j) {
      if (A[i] == A[j])
        return true;
    }
  }
  return false;
}
```

- Time complexity: $O(n^2)$
- Space complexity: $O(1)$

Exercise 1

```
int checkDuplicates(int A[], int n) {
  HeapSort(A, n);
  for (int i = 0; i < n - 1; ++i) {
    if (A[i] == A[i+1])
      return true;
  }
  return false;
}
```

- Time complexity: $O(nlogn)$
- Space complexity: $O(1)$

Exercise 2

- Given an array $A[0..n-1]$ where each element of the array represents a vote in the election.
- Assume that each vote is given as an integer representing the ID of the chosen candidate.
- Give an algorithm to determine who wins the election.
- Example
  - Input: [2, 3, 1, 2]
  - Output: 2

Exercise 2

```
int determineWinner(int A[], int n) {
  int i, j, candidate;
  int counter = 0, maxCounter = 0;
  candidate = A[0];
  for (int i = 0; i < n; ++i) {
    candidate = A[i];
    counter = 0;
    for (j = i + 1; j < n; ++j) {
      if (A[i] == A[j]) counter++;
    }
    if (counter > maxCounter) {
      maxCounter = counter;
      candidate = A[i];
    }
  }
  return candidate;
}
```

- Time complexity: $O(n^2)$
- Space complexity: $O(1)$

Exercise 2

```
int determineWinner(int A[], int n) {
    int i, j, currCand, maxCand, currCounter = 1, maxCounter = 1;
    Heapsort(A, n);
    currCand = maxCand = A[0];
    for (int i = 1; i < n; ++i) {
        if (A[i] == currCand) currCounter++;
        else {
            currCand = A[i];
            currCounter = 1;
        }
        if (currCounter > maxCounter){
            maxCounter = currCounter;
            maxCand = currCand;
        }
    }
    return maxCand;
}
```

- Time complexity: $O(nlogn)$
- Space complexity: $O(1)$